

색상 반전 2D 플랫폼어 - 개발 정리 문서

Godot 4.6.1 / Forward Plus / Jolt Physics

작성일: 2026-05-26

1. 게임 핵심 메커니즘

색상 규칙

플레이어 색	BLACK 지형	WHITE 지형	GRAY 지형
BLACK	✔️ 안전 (밟힘)	❌ 사망 (통과 후 판정)	⚠️ 미끄러짐
WHITE	❌ 사망 (통과 후 판정)	✔️ 안전 (밟힘)	⚠️ 미끄러짐

- 토글: Up 방향키로 BLACK ↔ WHITE 전환
- GRAY 지형: 올라갈 수 없음 (velocity.x = 0 강제). 단, 페인트로 색 칠하면 이동 가능해짐
- 페인트 총: 마우스 좌클릭으로 발사. 지형 표면 일부만 색이 바뀜 (전체 변환 아님)

2. 프로젝트 파일 구조

```
프로젝트ver-2/
├── main.tscn                ← 진입점 씬
├── project.godot
├── autoload/
│   └── SceneManager.gd      ← 씬 전환 전담 Autoload
├── scripts/
│   ├── main.gd              ← main.tscn 스크립트
│   ├── stage.gd             ← 스테이지 공통 스크립트
│   ├── player.gd            ← 플레이어 로직
│   ├── platform.gd          ← 발판 로직
│   ├── bullet.gd            ← 총알 로직
│   ├── paint_mark.gd        ← 페인트 얼룩 로직
│   └── color_defs.gd        ← 색 상수 전역 정의
├── scenes/
│   ├── player/Player.tscn
│   ├── platforms/Platform.tscn
│   ├── bullet/Bullet.tscn
│   ├── effects/PaintMark.tscn
│   └── world_1/stage_1/stage_1.tscn
└── .godot/ (자동생성, 수정 불필요)
```

3. 물리 레이어 구조 (project.godot)

레이어 번호	이름	비트 값	용도
Layer 1	player	$1 \ll 0 = 1$	플레이어 본체
Layer 2	platform_black	$1 \ll 1 = 2$	검정 발판 충돌
Layer 3	platform_white	$1 \ll 2 = 4$	흰색 발판 충돌
Layer 4	platform_gray	$1 \ll 3 = 8$	회색 발판 충돌
Layer 5	bullet	$1 \ll 4 = 16$	총알
Layer 6	paint_detector	$1 \ll 5 = 32$	PaintMark의 JudgmentZone
Layer 7	kill_zone	$1 \ll 6 = 64$	DeathDetector (발판 색 판정)

4. 씬별 노드 구조

Player.tscn

```
CharacterBody2D (player.gd)
├─ CollisionShape2D      ← 32x64 RectangleShape2D (물리 충돌)
├─ Sprite2D              ← 플레이어 비주얼 (현재 placeholder)
├─ GunPivot (Node2D)     ← 마우스를 향해 회전
│   └─ Muzzle (Marker2D) ← 총알 발사 기준점 (x=32)
├─ ColorSensor (Area2D)  ← mask=96(layer6+7), monitorable=false
│   └─ CollisionShape2D  ← 플레이어와 동일 크기
└─ Camera2D
```

Platform.tscn

```
StaticBody2D (platform.gd)
├─ CollisionPolygon2D     ← [역할1] 물리 충돌 (같은 색 플레이어만 뺄힘)
├─ DeathDetector (Area2D) ← layer=64, monitoring=false
│   └─ CollisionPolygon2D ← [역할2] 색 판정 감지 (ColorSensor가 탐지)
└─ Preview (Polygon2D)   ← 에디터 전용 비주얼
```

CollisionPolygon2D 두 개의 역할 정리

- **1번** (StaticBody2D 직속 자식): 물리적 충돌. color_state에 맞는 layer만 활성화되어 같은 색 플레이어만 뺄 수 있음
- **2번** (DeathDetector/Area2D 자식): 색 판정 감지 영역. ColorSensor가 이 Area2D를 감지해 사망 여부 결정

PaintMark.tscn

```
StaticBody2D (paint_mark.gd) ← 런타임에 collision_layer 설정
├─ CollisionShape2D          ← CircleShape2D r=28 (물리 충돌)
```

```

├─ JudgmentZone (Area2D)      ← layer=32 (paint_detector), monitoring=false
├─   └─ CollisionShape2D
└─ Sprite2D                  ← 페인트 얼룩 비주얼

```

Bullet.tscn

```

Area2D (bullet.gd)          ← layer=16(bullet), mask=14(black+white+gray)
├─ Sprite2D                 ← 총알 비주얼 (modulate로 색 표현)
├─ CollisionShape2D         ← CircleShape2D r=6
└─ LifeTimer (Timer)       ← wait=2.0, autostart=true (2초 후 자동 소멸)

```

5. 동작 흐름 (핵심)

5-1. 색 판정 (생사 판단) 흐름

```

[매 프레임 _physics_process]
├─ _check_color_death() 호출
│   └─ ColorSensor.get_overlapping_areas() 로 겹친 Area2D 목록 수집
│       └─ paint_marks 그룹 → JudgmentZone들 (우선순위 높음)
│           └─ paint_color == player_color ? 안전 : die()
└─ death_zones 그룹 → DeathDetector들 (페인트 없을 때만 판정)
    └─ plat.color_state != player_color ? die() : 안전

```

5-2. 총알 발사 흐름

```

[마우스 좌클릭 → _shoot()]
1. BULLET_SCENE.instantiate()
2. bullet.bullet_color = 반대색 (BLACK이면 WHITE 총알, 반대도 동일)
3. bullet.direction = (마우스 위치 - Muzzle 위치).normalized()
4. get_tree().current_scene.add_child(bullet) ← 씬 루트에 추가
5. bullet.global_position = Muzzle.global_position

```

5-3. 총알 충돌 → 페인트 생성 흐름

```

[총알 Area2D body_entered 신호]
├─ body가 "paint_bodies" 그룹?
│   └─ YES → body.call_deferred("update_color", bullet_color) ← 덮어쓰기
└─ NO (일반 지형)
    └─ _spawn_mark() 호출
        1. PaintMark.instantiate()
        2. mark.paint_color = bullet_color
        3. mark.global_position = 총알 위치 + direction * 6.0

```

```
4. parent.call_deferred("add_child", mark) ← 물리 flush 오류 방지
└─ _safe_free() 로 총알 소멸
```

5-4. PaintMark 초기화 흐름

```
[PaintMark _ready()]
1. add_to_group("paint_bodies") ← 총알이 덮어쓰기 가능하게
2. $JudgmentZone.add_to_group("paint_marks") ← ColorSensor가 우선 인식
3. _apply(paint_color) ← collision_layer + sprite.modulate 설정
4. _play_seep_tween() ← 스며드는 애니메이션 (scale 0.2→1.0, 0.12초)
```

5-5. 플레이어 색 전환 흐름

```
[Up 방향키 → _set_color(새 색)]
1. player_color 업데이트
2. collision_mask 업데이트
   - BLACK → mask = LAYER_BLACK | LAYER_GRAY (layer2 + layer4 = 10)
   - WHITE → mask = LAYER_WHITE | LAYER_GRAY (layer3 + layer4 = 12)
3. sprite.modulate 업데이트 (Color.BLACK / Color.WHITE)
※ collision_mask 변경으로 반대색 발판은 충돌 없이 통과, ColorSensor가 판정
```

5-6. 사망 및 리스폰 흐름

```
[die() 호출]
1. is_dead = true
2. velocity = Vector2.ZERO
3. died 시그널 emit (UI 연결 가능)
4. await 0.6초 (사망 모션 재생 시간, TODO)
5. _respawn()
   └─ global_position = respawn_point (현재 시작 위치 고정)
      is_dead = false
```

5-7. GRAY 지형 미끄러짐 흐름

```
[_physics_process 내]
1. _is_touching_gray() 체크
   └─ get_slide_collision(i).get_collider().color_state == GRAY ?
2. on_gray == true 이면:
   - floor_stop_on_slope = false ← 경사면에서 멈추지 않음
   - velocity.x = 0.0 ← 좌우 이동 불가
3. move_and_slide() 실행
※ PaintMark는 color_state 프로퍼티 없음 → GRAY 판정 안 됨 → 페인트 후 정상 이동
```

6. 씬 전환 흐름

```

F5 실행
├─ main.tscn 로드
│   └─ main.gd _ready()
│       └─ call_deferred("_start") ← 한 프레임 뒤 실행 (remove_child 충돌 방
지)
│           └─ DEBUG_STAGE > 0 ?
│               YES → SceneManager.load_stage(1)
│                   └─
change_scene_to_file("res://scenes/world_1/stage_1/stage_1.tscn")
│                   NO → SceneManager.go_to_main_menu() (미구현)

```

7. 해결된 주요 버그

#	버그	원인	해결
1	총알 색이 항상 흰색으로 보임	add_child 후 bullet_color 설정 → _ready()가 먼저 실행	add_child 전 에 bullet_color, direction 설정
2	Physics flush 오류	body_entered 콜백 내에서 add_child 직접 호출	<code>call_deferred("add_child", mark)</code> 사용
3	반대색 발판 위에서 사망 안 됨	PaintMark가 Area2D라 물리 충돌 없어 하부 발판에 닿아 판정 혼선	PaintMark를 StaticBody2D로 변경, JudgmentZone(Area2D)을 자식으로 분리
4	SceneManager remove_child 오류	_ready() 안에서 change_scene 직접 호출	<code>call_deferred("_start")</code> 패턴으로 수정
5	stage_1.tscn 삭제 후 재생성	.godot/editor/*.cfg 파일이 이전 경로 기억	editor_layout.cfg, project_metadata.cfg에서 참조 제거
6	타입 추론 Parser Error	<code>var matches :=</code> 같은 애매한 타입	<code>: bool, : Node</code> 등 명시적 타입 선언

8. 현재 TODO (미구현)

- ☐ 사망 모션 / 파티클 (die() 내 TODO 주석)
- ☐ 리스폰 포인트 노드화 (현재 시작 위치 고정)
- ☐ 총알/PaintMark 텍스처 파일 교체 (흰색총알.png, 검정총알.png)
- ☐ PaintDetector 추가 (DeathDetector와 별도 분리)
- ☐ GRAY 경사면 세부 조정
- ☐ 맵 일러스트(PNG) 적용 + CollisionPolygon2D 트레이싱
- ☐ MainMenu UI 구현
- ☐ StageSelect UI 구현
- ☐ 포탈 + 스테이지 클리어 시스템

- ❑ SceneManager STAGES 디렉터리에서 stage 2~30 경로 추가

9. 충돌 형태 선택 가이드 (CollisionPolygon2D vs 대안)

CollisionPolygon2D — 정점 제한 없음

기본 생성 시 사각형(4개)으로 보이지만, 정점은 무한정 추가 가능.

에디터 조작	기능
선분 위 클릭	새 정점 추가
정점 우클릭	정점 삭제
정점 드래그	위치 이동

⚠ 단점: 볼록(convex) 형태만 지원. 오목(concave, 구멍·굴곡 있는 형태)은 불가.

대안 비교

노드	형태	특징	사용 가능 바디
CollisionPolygon2D	볼록 다각형	정점 무제한, 에디터 편집 쉬움	모든 바디
CollisionShape2D + ConvexPolygonShape2D	볼록 다각형	CollisionPolygon2D와 유사	모든 바디
CollisionShape2D + ConcavePolygonShape2D	오목 다각형	구멍·굴곡 자유롭게 가능, 정점 무제한	StaticBody2D 전용
CollisionPolygon2D 여러 개 조합	복합 형태	복잡한 형태를 단순 폴리곤으로 분해	모든 바디

추천 용도

단순한 플랫폼 (직선·완만한 경사)

└─ CollisionPolygon2D 정점 추가

굴곡진 지형, 동굴, 복잡한 경사면

└─ CollisionShape2D + ConcavePolygonShape2D
(Platform.tscn은 StaticBody2D라 사용 가능)

움직이는 오브젝트 (플레이어, 적 등)

└─ CollisionPolygon2D 여러 개 조합
(CharacterBody2D에서는 ConcavePolygonShape2D 사용 불가)

🌟 맵 일러스트 트레이싱 시 권장:
ConcavePolygonShape2D → 가장 자유도 높음. StaticBody2D인 Platform에 바로 적용 가능.

10. 주요 설계 원칙

1. **플레이어가 주도하는 판정**: 발판이 플레이어를 감지하는 대신, 플레이어(ColorSensor)가 주변을 스캔 → 레이스 조건 방지
 2. **페인트 우선**: PaintMark(JudgmentZone)와 DeathDetector가 겹칠 때 paint_marks 그룹 먼저 판정
 3. **call_deferred 패턴**: 물리 콜백 내 실행 트리 변경은 항상 call_deferred 사용
 4. **add_child 전에 프로퍼티 설정**: 인스턴스화 후 _ready() 호출 전에 모든 설정 완료
 5. **ColorDefs class_name**: 색 상수를 단일 파일에서 관리, Autoload 불필요
-

이 문서는 Claude가 자동 생성했습니다. 프로젝트 루트에 저장: [프로젝트_개발정리.md](#)